



TEC | Tecnológico
de Costa Rica

Documentación Técnica - DonCE Kong Jr

Tarea 4 - CE 3104 Paradigmas de Programación

Integrantes:

M. Pamela Chacón Barrantes - 2023110706

Kevin Josué Ruiz Rodríguez - 2018170538

Franco Sagot López – 2022211182

Fecha de entrega: 28 noviembre, 2025

1.1. Descripción de las Estructuras de Datos Desarrolladas

1.1.1. Estructuras del Servidor (Java)

Clase Logic - Controlador Principal del Juego

java

```
public class Logic implements GameSubject {  
    private Integer level;  
    private Player player;  
    private ArrayList<Platform> platforms;  
    private ArrayList<Vine> vines;  
    private ArrayList<Collectible> collectibles;  
    private ArrayList<Enemy> enemies;  
    private Rectangle Boss;  
    private Rectangle Goal;  
    private ArrayList<GameObserver> observers;  
}
```

Descripción:

- **ArrayList<Platform> platforms:** Lista dinámica que almacena todas las plataformas del nivel. Utiliza ArrayList de Java para permitir acceso indexado $O(1)$ y agregar elementos en $O(1)$ amortizado.
- **ArrayList<Vine> vines:** Colección de lianas escalables. Cada liana es un rectángulo vertical donde el jugador puede trepar y los enemigos patrullar.
- **ArrayList<Collectible> collectibles:** Lista de frutas coleccionables (bananas, naranjas, fresas). Se itera en reversa durante la detección de colisiones para permitir eliminación segura.
- **ArrayList<Enemy> enemies:** Almacena los enemigos activos. Se limpia cuando el jugador pierde una vida o completa un nivel.
- **ArrayList<GameObserver> observers:** Implementación del patrón Observer. Contiene referencias a todos los espectadores conectados que reciben actualizaciones del estado del juego.

Justificación del uso de ArrayList:

- Acceso aleatorio eficiente para renderizado
- Iteración rápida para detección de colisiones
- Tamaño dinámico para agregar/eliminar entidades en runtime

Clase Entity - Estructura Base de Entidades

java

```
public abstract class Entity {  
  
    protected Double x;    // Posición X  
  
    protected Double y;    // Posición Y  
  
    protected Double vx;    // Velocidad en X  
  
    protected Double vy;    // Velocidad en Y  
  
    protected Integer width; // Ancho  
  
    protected Integer height; // Alto  
  
}
```

Descripción: Esta clase abstracta funciona como nodo base para todas las entidades del juego (jugador, enemigos, plataformas, coleccionables). Utiliza el patrón de herencia para compartir:

1. **Propiedades de transformación:** x, y, vx, vy permiten posicionamiento y física básica
2. **Propiedades de colisión:** width, height definen el bounding box
3. **Método getBounds():** Convierte las propiedades en un Rectangle de Java para detección de colisiones usando intersects()

Ventajas del diseño:

- **Polimorfismo:** Todas las entidades pueden tratarse uniformemente en colecciones
- **Reutilización de código:** La lógica de física y colisiones se hereda automáticamente

- **Extensibilidad:** Nuevos tipos de entidades solo necesitan extender esta clase

Estructura de Comunicación - GameClientHandler

java

```
public class GameClientHandler implements Runnable {  
    private Socket client;  
    private DataOutputStream output;  
    private DataInputStream input;  
    private Logic game;  
    private Integer gameNumber;  
    private Boolean connected;  
    private DisconnectCallback disconnectCallback;  
}
```

Descripción: Esta estructura actúa como un nodo de comunicación entre un cliente jugador y una instancia de juego:

- **Socket client:** Conexión TCP con el cliente
- **DataInputStream/DataOutputStream:** Streams para comunicación bidireccional usando el protocolo UTF de Java
- **Logic game:** Referencia a la instancia del juego que controla este cliente
- **DisconnectCallback:** Interface funcional para notificar desconexiones (patrón Callback)

Flujo de datos:

1. Thread de entrada: Lee comandos del cliente (move_left, jump, etc.)
2. Thread principal: Actualiza física del juego (30 FPS) y envía estado serializado
3. Notificación Observer: Propaga el estado a todos los espectadores

1.1.2. Estructuras del Cliente (C)

Estructura Player - Estado del Jugador

```
typedef struct {  
    float x;        // Posición en pantalla  
    float y;  
    float offsetX;  // Offset para suavizado visual  
    float offsetY;  
    float size;     // Tamaño de colisión  
    int lives;      // Vidas restantes  
    int score;      // Puntuación acumulada  
    int spriteIndex; // Índice del sprite actual (0-13)  
    int animCounter; // Contador de frames de animación  
    bool facingRight; // Dirección de orientación del sprite  
} Player;
```

Descripción: Estructura estática global (singleton) que mantiene el estado visual y lógico del jugador:

- **Posicionamiento:** x, y representan coordenadas en el espacio de pantalla (224x256)
- **Offsets:** offsetX, offsetY permiten ajustar visualmente el sprite respecto a la hitbox
- **Animación:** spriteIndex selecciona el frame correcto del spritesheet (14 frames de 32x16)
- **animCounter:** Contador que se incrementa cada frame para controlar la velocidad de animación (cambia sprite cada 8 frames)

Sincronización con servidor: La función `updatePlayerFromServer()` recibe datos del servidor y actualiza esta estructura, actuando como fuente única de verdad del estado del jugador.

Estructura Enemy - Array de Enemigos

```
typedef struct {  
    float x, y;      // Posición del enemigo  
    float velX, velY; // Velocidad (legacy, no usada)  
    float speed;     // Velocidad base  
    float size;      // Tamaño (16x16)  
    EnemyType type;  // RED (patrulla) o BLUE (gravedad)  
    int animFrame;   // Frame de animación (0-1)  
    bool active;     // ¿Está activo en el juego?  
    bool movingUp;   // Dirección vertical (para flip)  
} Enemy;
```

static Enemy enemies[MAX_ENEMIES] = {0}; // Array estático, max 10 enemigos

Descripción: Array estático de tamaño fijo que actúa como pool de objetos para enemigos:

- **Ventaja:** No requiere allocación dinámica (malloc/free), evitando fragmentación de memoria
- **active flag:** Implementa el patrón "object pooling" - los slots se reutilizan marcando activo/inactivo
- **EnemyType:** Enum que determina comportamiento (rojo = patrulla, azul = cae por gravedad)
- **movingUp:** Determina si el sprite debe voltearse verticalmente (enemigos rojos)

Algoritmo de actualización:

```
void updateEnemiesFromServer(ServerEnemyData* serverEnemies, int count) {  
    // 1. Desactivar todos los enemigos existentes  
    for (int i = 0; i < MAX_ENEMIES; i++) {
```

```

        enemies[i].active = false;
    }

    // 2. Activar solo los enemigos que el servidor reporta
    for (int i = 0; i < count && i < MAX_ENEMIES; i++) {
        if (serverEnemies[i].active) {
            enemies[i] = serverEnemies[i]; // Copia directa
            enemies[i].active = true;
        }
    }
}

```

Estructura Fruit - Coleccionables

```

typedef struct {
    float x, y;        // Posición en la liana
    float size;        // Tamaño (16x16)
    FruitType type;    // ORANGE, BANANA, STRAWBERRY
    bool active;       // Estado de activación
} Fruit;

static Fruit fruits[MAX_FRUITS] = {0}; // Array estático, max 20 frutas

```

Descripción: Similar a Enemy, usa un array estático con object pooling:

- **FruitType:** Enum que determina qué frame del spritesheet renderizar (0=naranja, 1=banana, 2=fresa)
- **Sincronización:** Se actualiza completamente desde el servidor cada frame (estado autoritativo en servidor)

Estructura NetworkConnection - Estado de Red

```

typedef struct {
    int sockfd;        // File descriptor del socket TCP
    bool connected;    // Estado de conexión
}

```

```
char serverAddress[256]; // "localhost" o IP  
  
int serverPort; // 2121  
  
} NetworkConnection;  
  
static NetworkConnection connection = { ... }; // Singleton global
```

Descripción: Estructura singleton que mantiene el estado de la conexión TCP:

- **sockfd:** En Windows es SOCKET, en Linux/Mac es un entero (file descriptor)
- **Socket no bloqueante:** Configurado con O_NONBLOCK (Linux) o FIONBIO (Windows) para permitir polling sin bloquear el game loop
- **Gestión de vida:** Se inicializa en connectToServer(), se limpia en disconnectFromServer()

1.1.3. Estructuras de Comunicación - Protocolo de Red

Formato de Mensaje del Servidor

text

STATE|<gameNum>|PLAYER|<x>,<y>,<lives>,<score>,<state>,<facing>|ENEMIES|<enemyList>|FRUITS|<fruitList>

Estructura de parsing:

```
typedef struct {  
    float x, y;  
  
    int lives, score;  
  
    const char* state; // "idle", "walking", "jumping", "climbing"  
    const char* facing; // "left", "right"  
} ServerPlayerData;  
  
typedef struct {  
    float x, y;  
  
    EnemyType type; // Parseado de "redenemy" o "blueenemy"  
  
    bool active;  
  
    bool movingUp; // Parseado de "up" o "down"  
} ServerEnemyData;
```

1.2. Descripción Detallada de los Algoritmos Desarrollados

1.2.1. Algoritmo de Parseo del Estado del Juego (Cliente C)

Ubicación: game_logic.c::parseGameState()

Propósito: Convertir el mensaje serializado del servidor Java en estructuras C utilizables.

Complejidad: $O(n)$ donde n es la longitud del mensaje

Pseudocódigo:

text

ALGORITMO parseGameState(data, dataLength)

ENTRADA:

- data: buffer de caracteres con mensaje del servidor
- dataLength: longitud del buffer

SALIDA: Actualización de estructuras globales (player, enemies, fruits)

INICIO

// Paso 1: Remover prefijo UTF-8 de Java (2 bytes de longitud)

SI dataLength $\geq$ 2 ENTONCES

 utfLength = (data[0] $\ll$ 8) | data[1]

 SI utfLength == dataLength - 2 ENTONCES

 data = data + 2 // Saltar prefijo

 dataLength = utfLength

 FIN SI

FIN SI

// Paso 2: Copiar a buffer local y añadir null terminator

copiar data[0..dataLength-1] a buffer[]

buffer[dataLength] = '\0'

```
// Paso 3: Verificar que sea mensaje STATE
SI buffer NO comienza con "STATE|" ENTONCES
    retornar ERROR
FIN SI
```

```
// Paso 4: Parsear sección PLAYER
playerSection = strstr(buffer, "|PLAYER|")
SI playerSection != NULL ENTONCES
    playerSection += 8 // Saltar "|PLAYER|"
    playerEnd = strchr(playerSection, '|')
    SI playerEnd != NULL ENTONCES
        extraer substring playerData
        // Parsear valores separados por comas
        sscanf(playerData, "%f,%f,%d,%d,%s,%s",
            &x, &y, &lives, &score, &state, &facing)
        // Crear estructura y actualizar jugador
        serverPlayer = {x, y, lives, score, state, facing}
        updatePlayerFromServer(&serverPlayer)
    FIN SI
FIN SI
```

```
FIN SI
// Paso 5: Parsear sección ENEMIES
enemiesSection = strstr(buffer, "|ENEMIES|")
SI enemiesSection != NULL ENTONCES
    enemiesSection += 9 // Saltar "|ENEMIES|"
    enemiesEnd = strchr(enemiesSection, '|')
```

SI enemiesEnd != NULL ENTONCES

extraer substring enemiesData

SI enemiesData está vacío ENTONCES

// No hay enemigos, limpiar array

updateEnemiesFromServer(NULL, 0)

SINO

// Parsear lista: "redenemy(x,y,dir);blueenemy(x,y,dir);..."

enemyCount = 0

ptr = enemiesData

MIENTRAS *ptr != '\0' Y enemyCount < MAX_ENEMIES

// Saltar separadores

MIENTRAS *ptr == ';' O *ptr == ' '

ptr++

FIN MIENTRAS

// Encontrar paréntesis

paren = strchr(ptr, '(')

SI paren == NULL ENTONCES salir

// Extraer tipo (texto antes del paréntesis)

typeLen = paren - ptr

copiar ptr[0..typeLen-1] a type[]

// Parsear coordenadas y dirección

sscanf(paren, "(%f,%f,%s)", &x, &y, &dir)

// Crear estructura de enemigo

```

serverEnemies[enemyCount] = {
    x, y,
    tipo_desde_string(type),
    true,
    (dir == "up")
}
enemyCount++

// Avanzar al siguiente enemigo
ptr = strchr(ptr, ';')

SI ptr != NULL ENTONCES ptr++

SINO salir

FIN MIENTRAS

// Actualizar array de enemigos
updateEnemiesFromServer(serverEnemies, enemyCount)

FIN SI

FIN SI

FIN SI

// Paso 6: Parsear sección FRUITS (similar a ENEMIES)
fruitsSection = strstr(buffer, "|FRUITS|")

SI fruitsSection != NULL ENTONCES

    // ... (proceso análogo al parsing de enemigos)

    // Formato: "orange(x,y);banana(x,y);strawberry(x,y)"

FIN SI

FIN

```

Consideraciones de implementación:

1. **Manejo de prefijo UTF:** Java's `DataOutputStream.writeUTF()` añade 2 bytes de longitud en big-endian
2. **Parsing robusto:** Usa `strstr()` para encontrar secciones, evitando asumir posiciones fijas
3. **Separadores múltiples:** Permite ; y espacios como separadores
4. **Validación:** Verifica que los índices no excedan límites de arrays

1.3.2. Algoritmo de Detección de Colisiones (Servidor Java)

Ubicación: Logic.java::collisions()

Propósito: Detectar y resolver colisiones entre el jugador y el entorno (plataformas, lianas, enemigos, coleccionables).

Complejidad: $O(p + v + e + c)$ donde p =plataformas, v =lianas, e =enemigos, c =coleccionables

Pseudocódigo detallado:

text

ALGORITMO collisions()

ENTRADA: Estado actual del jugador y entidades del juego

SALIDA: Actualización de posiciones y estados tras resolver colisiones

INICIO

// ===== FASE 1: INICIALIZACIÓN =====

playerRect = jugador.getBounds() // Rectangle(x, y, width, height)

jugador.onGround = falso

wasClimbing = jugador.climbing

jugador.climbing = falso

// ===== FASE 2: COLISIÓN CON LIANAS =====

isJumping = (jugador.vy < 0) Y NO wasClimbing

wantsToClimb = (jugador.currentDirection == "up" O jugador.currentDirection == "down")

PARA CADA vine EN vines HACER

vineRect = vine.getBounds()

SI playerRect.intersects(vineRect) Y wantsToClimb Y NO isJumping ENTONCES

// Jugador agarra la liana

jugador.climbing = verdadero

// Centrar jugador en la liana

jugador.x = vine.x + vine.width/2 - jugador.width/2

// Si recién agarró la liana, detener momentum vertical

SI NO wasClimbing ENTONCES

jugador.vy = 0.0

FIN SI

salir del bucle

FIN SI

FIN PARA

// ===== FASE 3: COLISIÓN CON PLATAFORMAS =====

PARA CADA platform EN platforms HACER

platformRect = platform.getBounds()

SI playerRect.intersects(platformRect) ENTONCES

// Calcular overlaps en cada dirección

overlapLeft = (jugador.x + jugador.width) - platform.x

overlapRight = (platform.x + platform.width) - jugador.x

overlapTop = (jugador.y + jugador.height) - platform.y

overlapBottom = (platform.y + platform.height) - jugador.y


```
// Encontrar el overlap mínimo (dirección de colisión más probable)
minOverlap = min(overlapLeft, overlapRight, overlapTop, overlapBottom)
```

```
// Resolver colisión según la dirección de menor overlap
```

```
SI minOverlap == overlapTop Y jugador.vy >= 0 ENTONCES
```

```
    // Colisión desde arriba (jugador aterriza en plataforma)
```

```
    jugador.y = platform.y - jugador.height
```

```
    jugador.vy = 0.0
```

```
    jugador.onGround = verdadero
```

```
    jugador.jumping = falso
```

```
SINO SI minOverlap == overlapBottom Y jugador.vy <= 0 ENTONCES
```

```
    // Colisión desde abajo (jugador golpea techo)
```

```
    jugador.y = platform.y + platform.height
```

```
    jugador.vy = 0.0
```

```
SINO SI minOverlap == overlapLeft Y jugador.vx > 0 ENTONCES
```

```
    // Colisión lateral izquierda
```

```
    jugador.x = platform.x - jugador.width
```

```
    jugador.vx = 0.0
```

```
SINO SI minOverlap == overlapRight Y jugador.vx < 0 ENTONCES
```

```
    // Colisión lateral derecha
```

```
    jugador.x = platform.x + platform.width
```

```
    jugador.vx = 0.0
```

FIN SI

FIN SI

FIN PARA

// ===== FASE 4: VERIFICACIÓN ADICIONAL DE SUELO =====

// Para casos donde el jugador está muy cerca de una plataforma

SI NO jugador.onGround Y NO jugador.climbing Y jugador.vy >= 0 ENTONCES

PARA CADA platform EN platforms HACER

playerBottom = jugador.y + jugador.height

platformTop = platform.y

// Verificar overlap horizontal

horizontalOverlap = (jugador.x + jugador.width > platform.x) Y

(jugador.x < platform.x + platform.width)

// Distancia vertical al tope de la plataforma

distanceToTop = playerBottom - platformTop

// Si está dentro de 2 píxeles y hay overlap horizontal

SI horizontalOverlap Y distanceToTop >= 0 Y distanceToTop <= 2 ENTONCES

jugador.y = platformTop - jugador.height

jugador.vy = 0.0

jugador.onGround = verdadero

jugador.jumping = falso

salir del bucle

FIN SI

FIN PARA

FIN SI

// ===== FASE 5: COLISIÓN CON ENEMIGOS =====

hitEnemy = falso

PARA CADA enemy EN enemies HACER

 enemyRect = enemy.getBounds()

 SI playerRect.intersects(enemyRect) ENTONCES

 hitEnemy = verdadero

 log("Player hit enemy at (" + enemy.x + ", " + enemy.y + ")")

 salir del bucle

 FIN SI

FIN PARA

SI hitEnemy ENTONCES

 log("Lives remaining: " + (jugador.lives - 1))

 jugador.lives -= 1

 enemies.clear()

 collectibles.clear()

 setPlayer() // Resetear posición del jugador

FIN SI

// ===== FASE 6: COLISIÓN CON BOSS O CAÍDA =====

SI playerRect.intersects(Boss) O jugador.y + jugador.height > 256 ENTONCES

```
jugador.lives -= 1
```

```
enemies.clear()
```

```
collectibles.clear()
```

```
setPlayer()
```

```
FIN SI
```

```
// ===== FASE 7: COLISIÓN CON COLECCIONABLES =====
```

```
PARA i = collectibles.size() - 1 HASTA 0 (en reversa) HACER
```

```
    collectible = collectibles.get(i)
```

```
    collectibleRect = collectible.getBounds()
```

```
    SI playerRect.intersects(collectibleRect) ENTONCES
```

```
        jugador.score += collectible.value
```

```
        collectibles.remove(i) // Remover del final evita
```

```
        ConcurrentModificationException
```

```
    FIN SI
```

```
FIN PARA
```

```
// ===== FASE 8: COLISIÓN CON OBJETIVO (VICTORIA) =====
```

```
SI playerRect.intersects(Goal) ENTONCES
```

```
    SI jugador.lives < 6 ENTONCES
```

```
        jugador.lives += 1
```

```
    FIN SI
```

```
    level += 1
```

```
    enemies.clear()
```

```
    collectibles.clear()
```

setPlayer()

FIN SI

FIN

Análisis de complejidad:

- **Mejor caso:** $O(p + v)$ - solo chequea plataformas y lianas
- **Peor caso:** $O(p + v + e + c)$ - itera todas las entidades
- **Optimización posible:** Implementar spatial partitioning (quadtree) para reducir a $O(\log n)$

Características importantes:

1. **Resolución por overlap mínimo:** Previene que el jugador se "atasque" en esquinas
2. **Iteración reversa en coleccionables:** Evita errores al remover elementos durante iteración
3. **Flag hitEnemy:** Previene ConcurrentModificationException al modificar la lista fuera del bucle
4. **Verificación adicional de suelo:** Maneja casos edge donde la física discreta puede fallar

1.3.3. Algoritmo de Aplicación de Gravedad (Servidor Java)

Ubicación: Player.java::applyGravity()

Propósito: Simular gravedad realista con aceleración y velocidad terminal.

Complejidad: $O(1)$

Pseudocódigo:

text

ALGORITMO aplicar_gravedad()

ENTRADA: Estado actual del jugador (onGround, climbing, vy)

SALIDA: vy actualizado

1. SI !onGround Y !climbing ENTONCES:

a. gravity = 0.5

b. vy = vy + gravity

c. maxFallSpeed = 12.0

d. SI vy > maxFallSpeed ENTONCES:

- vy = maxFallSpeed

FIN SI

FIN SI

FIN ALGORITMO

Detalles de implementación:

- **Aceleración constante:** gravity = 0.5 píxeles/frame²
- **Velocidad terminal:** maxFallSpeed = 12.0 píxeles/frame (evita caídas infinitamente rápidas)
- **Condicionales de activación:** Solo aplica cuando no está en suelo ni escalando

Ecuación de movimiento:

$$v_y(t+1) = \min(v_y(t) + 0.5, 12.0)$$

$$y(t+1) = y(t) + v_y(t+1)$$

Algoritmo 3: Movimiento del Jugador (Player.move(String direction))

Propósito: Actualizar velocidad del jugador según input del usuario.

Pseudocódigo:

text

ALGORITMO mover_jugador(direccion)

ENTRADA: String direccion ("left", "right", "up", "down", "jump")

SALIDA: vx y vy actualizados

1. moveSpeed = 3.0

2. climbSpeed = 2.0

3. currentDirection = direccion

4. SI direccion == "left" O "right" ENTONCES:

- lastHorizontalDirection = direccion

FIN SI

5. CASO direccion DE:

"up":

SI climbing ENTONCES:

vy = -climbSpeed

FIN SI

"down":

SI climbing ENTONCES:

vy = climbSpeed

FIN SI

"left":

vx = -moveSpeed

"right":

vx = moveSpeed

"jump":

SI onGround O climbing ENTONCES:

- vy = -5.0
- onGround = false
- climbing = false
- jumping = true

FIN SI

DEFECTO:

LLAMAR stop()

FIN CASO

FIN ALGORITMO

Características:

- **Velocidad constante:** No hay aceleración en movimiento horizontal (juego arcade clásico)
- **Separación de ejes:** Movimiento horizontal y vertical independientes
- **Salto desde liana:** Permite saltar mientras escala
- **Memoria de dirección:** Guarda última dirección horizontal para renderizado de sprite

Algoritmo 4: Patrullaje de Enemigo Rojo (RedEnemy.patrol())

Propósito: Crear movimiento oscilatorio vertical dentro de los límites de la liana.

Pseudocódigo:

text

ALGORITMO patrullar_enemigo_rojo()

ENTRADA: Posición actual (y), velocidad (vy), referencia a liana (vine)

SALIDA: vy actualizado (dirección invertida si toca límite)

1. SI $y + \text{height} > \text{vine.y} + \text{vine.height}$ ENTONCES:

- $vy = -vy$ // Invertir dirección (rebote en límite inferior)

FIN SI

2. SI NO, SI $y < \text{vine.y}$ ENTONCES:

- $vy = -vy$ // Invertir dirección (rebote en límite superior)

FIN SI

FIN ALGORITMO

Comportamiento:

- Movimiento sinusoidal dentro de la liana
- Inversión instantánea de velocidad al tocar límites
- No hay pérdida de energía (rebote perfectamente elástico)

Algoritmo 5: Patrullaje de Enemigo Azul (BlueEnemy.patrol())

Propósito: Simular enemigo que cae con gravedad dentro de la liana.

Pseudocódigo:

text

ALGORITMO `patrullar_enemigo_azul()`

ENTRADA: Posición actual (y), velocidad (vy), referencia a liana (vine)

SALIDA: vy actualizado con gravedad

1. SI $y + \text{height} > \text{vine.y} + \text{vine.height}$ ENTONCES:

a. $\text{gravity} = 0.5$

b. $\text{vy} = \text{vy} + \text{gravity}$

c. $\text{maxFallSpeed} = 12.0$

d. SI $\text{vy} > \text{maxFallSpeed}$ ENTONCES:

- $\text{vy} = \text{maxFallSpeed}$

FIN SI

FIN SI

FIN ALGORITMO

Diferencia con enemigo rojo:

- Aplica gravedad constante (igual que el jugador)
- No rebota en límites inferiores (solo acelera hacia abajo)
- Más impredecible y desafiante

Algoritmo 6: Construcción del Estado del Juego

(GameClientHandler.buildGameState())

Propósito: Serializar el estado completo del juego en un string para transmisión por red.

Pseudocódigo:

text

ALGORITMO `construir_estado_juego()`

ENTRADA: game (instancia de Logic con todos los datos)

SALIDA: String con estado serializado

1. Crear StringBuilder gameState

2. Agregar "STATE|"

3. Agregar gameNumber + "|"

4. Construir sección PLAYER:

a. Agregar "PLAYER|"

b. Agregar player.x + ","

c. Agregar player.y + ","

d. Agregar player.lives + ","

e. Agregar player.score + ","

f. Agregar player.state + ","

g. Agregar player.facingDirection + "|"

5. Construir sección ENEMIES:

a. Agregar "ENEMIES|"

b. Llamar buildEnemiesList() y agregar resultado + "|"

6. Construir sección FRUITS:

a. Agregar "FRUITS|"

b. Llamar buildFruitsList() y agregar resultado

7. Retornar gameState.toString()

FIN ALGORITMO

ALGORITMO construir_lista_enemigos()

ENTRADA: Lista de enemigos activos

SALIDA: String con formato "tipo(x,y,dir);tipo(x,y,dir);..."

1. Crear StringBuilder enemies

2. PARA i = 0 HASTA enemyList.size() - 1 HACER:

a. enemy = enemyList[i]

b. type = enemy.getClass().getSimpleName().toLowerCase()

c. direction = (enemy.vy < 0) ? "up" : "down"

d. Agregar type + "(" + enemy.x + "," + enemy.y + "," + direction + ")"

e. SI i < enemyList.size() - 1 ENTONCES:

- Agregar ";"

FIN SI

FIN PARA

3. Retornar enemies.toString()

FIN ALGORITMO

ALGORITMO construir_lista_frutas()

ENTRADA: Lista de coleccionables activos

SALIDA: String con formato "tipo(x,y);tipo(x,y);..."

1. Crear StringBuilder fruits

2. PARA i = 0 HASTA collectibleList.size() - 1 HACER:

a. collectible = collectibleList[i]

b. `type = collectible.getClass().getSimpleName().toLowerCase()`

c. Agregar `type + "(" + collectible.x + ";" + collectible.y + ")"`

d. Si `i < collectibleList.size() - 1` ENTONCES:

- Agregar `","`

FIN SI

FIN PARA

3. Retornar `fruits.toString()`

FIN ALGORITMO

Ejemplo de salida:

```
STATE|1|PLAYER|100.5,200.0,3,450,walking,right|ENEMIES|redenemy(50.0,100.0,up);
blueenemy(80.0,150.0,down)|FRUITS|banana(60.0,120.0);orange(90.0,140.0);strawb
erry(110.0,160.0)
```

Complejidad:

- $O(e + c)$ donde e = enemigos, c = coleccionables
- `StringBuilder` evita concatenación ineficiente de Strings

1.3. Problemas sin solución

1.3.1. Sincronización de Animaciones de Enemigos en Múltiples Clientes

Descripción del problema: Las animaciones de los enemigos no están completamente sincronizadas entre diferentes clientes espectadores y jugadores. Cada cliente mantiene su propio contador de animación local (enemyAnimCounter en render.c), lo que causa que los frames de animación de los cocodrilos no coincidan exactamente entre diferentes instancias del cliente.

Impacto:

- Bajo impacto en jugabilidad (la posición sí está sincronizada)
- Inconsistencia visual entre clientes
- Dificulta la depuración de problemas relacionados con enemigos

Razón de no solución: Implementar sincronización de frames de animación requeriría:

1. Modificar el protocolo de comunicación para incluir frame de animación
2. Aumentar significativamente el ancho de banda (enviar frame adicional por cada enemigo)
3. Sincronización de reloj entre servidor y clientes
4. El beneficio no justifica la complejidad añadida

Workaround actual: Las animaciones se ejecutan de forma independiente en cada cliente, lo cual es aceptable ya que la posición y dirección sí están sincronizadas.

1.3.2. Reconexión Automática tras Pérdida de Conexión

Descripción del problema: Si se pierde la conexión con el servidor durante una partida (por ejemplo, si el servidor se cierra inesperadamente), el cliente no intenta reconectarse automáticamente. El usuario debe volver al menú principal y reiniciar la conexión manualmente.

Impacto:

- Pérdida de progreso de la partida actual
- Experiencia de usuario interrumpida

- No hay sistema de recuperación de sesión

Razón de no solución: Implementar reconexión automática requeriría:

1. Sistema de detección de latidos (heartbeat) entre cliente y servidor
2. Buffer de comandos pendientes durante la desconexión
3. Mecanismo de restauración de estado del juego en el servidor
4. Manejo de casos edge (ej: ¿qué pasa si el slot del juego fue tomado por otro jugador?)
5. Tiempo de desarrollo significativo para un feature que es más crítico en juegos con sesiones largas

Workaround actual: El cliente detecta la desconexión y retorna al menú principal, donde el usuario puede intentar reconectarse manualmente.

1.3.3. Interpolación de Movimiento para Reducir "Jitter"

Descripción del problema: Debido a que el servidor envía actualizaciones a ~30 FPS y la red puede introducir latencia variable, ocasionalmente se observa un efecto de "jitter" o movimiento entrecortado, especialmente notable en espectadores con conexiones de mayor latencia.

Impacto:

- Movimiento no completamente suave en algunas condiciones de red
- Más notable en modo espectador que en modo jugador
- No afecta la jugabilidad real, solo la percepción visual

Razón de no solución: Implementar interpolación requeriría:

1. Buffer de estados históricos (mínimo 2-3 frames)
2. Algoritmo de interpolación lineal o curva entre posiciones
3. Sincronización de timestamp entre cliente y servidor
4. Mayor complejidad en el código de renderizado
5. Posible introducción de input lag percibido en modo jugador
6. El juego corre a 30 FPS de forma deliberada para mantener la estética retro

Workaround actual: El cliente renderiza las posiciones exactas recibidas del servidor. A 30 FPS y en una red local (localhost), el efecto es mínimo y aceptable.

1.3.4. Colisiones Predictivas en el Cliente

Descripción del problema: Todas las colisiones se calculan en el servidor (arquitectura autoritativa), lo que significa que hay un ligero delay entre cuando el jugador presiona una tecla y cuando ve el resultado de una colisión (especialmente colisión con frutas o enemigos).

Impacto:

- Input lag de ~32-64ms en colisiones
- Sensación de menor "responsividad" comparado con juegos puramente locales
- Ocasionalmente el jugador visualmente "toca" un objeto antes de que el servidor confirme la colisión

Razón de no solución: Implementar predicción de colisiones en el cliente requeriría:

1. Duplicar toda la lógica de física y colisiones en el cliente C
2. Sistema de reconciliación cuando la predicción difiere del servidor
3. Manejo de correcciones visuales ("rubber banding")
4. Mayor complejidad y posibilidad de bugs de sincronización
5. Mantener dos implementaciones paralelas de la misma lógica

Decisión de diseño: Se optó por una arquitectura completamente autoritativa donde el servidor es la única fuente de verdad. Esto simplifica enormemente el código, elimina posibilidades de cheating, y el lag de 32-64ms es aceptable para un juego casual de estética retro.

1.5. Plan de Actividades Realizadas por Estudiante

#	Tarea	Descripción	Tiempo Estimado	Responsable	Fecha Inicio
1	Análisis de Arquitectura	Diseño de arquitectura cliente-servidor, definición de protocolo de comunicación, diagramas de clases	2 horas	TODOS	10-Nov-2025
2	Implementación Server Base	Clase Server.java, manejo de sockets TCP, sistema de IDs para clientes	6 horas	KEVIN	11-Nov-2025
3	Sistema de Lógica del Juego	Implementación de Logic.java: física, colisiones, gestión de entidades	6 horas	KEVIN	12-Nov-2025
4	Patrón Factory	Implementación de factories para enemigos y coleccionables, clases concretas	4 horas	PAMELA	11-Nov-2025

#	Tarea	Descripción	Tiempo Estimado	Responsable	Fecha Inicio
5	GameClientHandler	Manejo de jugadores individuales, serialización de estado, loop de juego	8 horas	PAMELA	12-Nov-2025
6	Patrón Observer	Interfaces GameSubject y GameObserver , implementación en Logic.java	4 horas	FRANCO	13-Nov-2025
7	SpectatorHandler	Sistema de espectadores usando Observer, diferenciación player/spectator	3 horas	FRANCO	14-Nov-2025
8	Multi-Game Support	Sistema de 2 juegos simultáneos en App.java, gestión de slots	3 horas	FRANCO	14-Nov-2025
9	Cliente C - Setup SDL3	Inicialización de SDL3, ventana, render, etc.	6 horas	FRANCO	15-Nov-2025

#	Tarea	Descripción	Tiempo Estimado	Responsable	Fecha Inicio
		sistema de assets			
10	Sistema de Red en C	network.c/h: sockets TCP en C, soporte Windows/Linux , comunicación con Java	15 horas	PAMELA	13-Nov-2025
11	Parseo de Protocolo	Implementación en game_logic.c para parsear mensajes del servidor (formato UTF Java)	4 horas	PAMELA	14-Nov-2025
12	Sistema de Renderizado	render.c: menús, HUD, sprites, animaciones, diferentes estados del juego	9 horas	FRANCO	17-Nov-2025
13	Sistema de Input	input.c: manejo de eventos SDL, envío de comandos al servidor	4 horas	FRANCO	19-Nov-2025

#	Tarea	Descripción	Tiempo Estimado	Responsable	Fecha Inicio
14	Assets y Sprites	Carga de texturas BMP, spritesheets, fuentes TTF, configuración de color keys	6 horas	FRANCO	19-Nov-2025
15	Game Over Screen	Pantalla de game over, manejo de vidas negativas, flujo de retorno al menú	4 horas	PAMELA	17-Nov-2025
16	Testing y Debugging	Pruebas de sincronización, detección de memory leaks, corrección de bugs	12 horas	KEVIN	20-Nov-2025
17	Documentación	Comentarios en código, documentación de protocolo, README	6 horas	PAMELA	28-Nov-2025
18	Refinamiento Final	Ajustes de balance, pulido de animaciones, optimizaciones	4 horas	TODOS	28-Nov-2025

1.6. Problemas Encontrados

1.6.1. Formato UTF de Java en Comunicación de Red

Descripción Detallada del Problema

Al implementar la comunicación entre el servidor Java y el cliente C, se encontró un problema crítico: los mensajes enviados desde Java usando `DataOutputStream.writeUTF()` no podían ser parseados correctamente en el cliente C.

Síntomas observados:

text

Cliente C recibe: [00 0D] H e l l o W o r l d

^^^^^^

2 bytes extra misteriosos

El cliente intentaba procesar los primeros 2 bytes como parte del mensaje, resultando en caracteres basura o crashes al intentar parsear comandos.

Root cause: El método `writeUTF()` de Java no envía un simple string UTF-8, sino que utiliza un formato modificado de UTF-8 con un prefijo de 2 bytes que indica la longitud del string:

Formato Java `writeUTF`:

text

[Byte 1: High byte of length][Byte 2: Low byte of length][UTF-8 data...]

Ejemplo: "Hello" (5 caracteres)

text

[0x00][0x05][0x48][0x65][0x6C][0x6C][0x6F]

Solución Encontrada

Se implementó un enfoque híbrido que maneja el prefijo de longitud en dos lugares diferentes del código:

1. En el envío (Cliente → Servidor):

```
// network.c - CORRECTO
```

```

bool sendCommandToServer(const char* command) {
    size_t commandLen = strlen(command);
    if (commandLen > 65535) {
        SDL_Log("Comando demasiado largo");
        return false;
    }
    // Crear buffer con prefijo de longitud (formato Java UTF)
    unsigned char buffer[65538];
    buffer[0] = (commandLen >> 8) & 0xFF; // High byte
    buffer[1] = commandLen & 0xFF;      // Low byte
    memcpy(buffer + 2, command, commandLen);

    // Enviar con prefijo de longitud
    return sendToServer((const char*)buffer, commandLen + 2);
}

```

2. En la recepción (Servidor → Cliente):

c

// game_logic.c - CORRECTO

```

static void parseGameState(const char* data, int dataLength) {
    const char* actualData = data;
    int actualLength = dataLength;

    // Detectar y saltar el prefijo UTF si existe
    if (dataLength >= 2) {
        unsigned char byte1 = (unsigned char)data[0];
        unsigned char byte2 = (unsigned char)data[1];
    }
}

```

```

unsigned short utfLength = (byte1 << 8) | byte2;

// Si la longitud UTF coincide con los datos restantes,
// es un prefijo válido
if (utfLength == dataLength - 2) {
    actualData = data + 2;
    actualLength = utfLength;
}
}

// Continuar con el parseo del mensaje...

char buffer[4096];

int copyLen = actualLength < sizeof(buffer) - 1 ?
    actualLength : sizeof(buffer) - 1;
strncpy(buffer, actualData, copyLen);
buffer[copyLen] = '\0';

// Parsear el estado del juego...
}

```

Ventajas de esta solución:

- Simple y directa
- No requiere estado entre llamadas
- Compatible con socket no-bloqueante
- Maneja correctamente mensajes fragmentados (TCP los reensambla)

1.6.2. Gestión de Estados de Conexión y Desconexión

Descripción Detallada del Problema

El manejo de conexiones y desconexiones en un juego multiplayer es complejo debido a múltiples factores:

1. Desconexión limpia vs. abrupta: El cliente puede desconectarse intencionalmente (ESC) o inesperadamente (crash, red caída)
2. Sincronización de slots: El servidor debe liberar el slot del juego cuando un jugador se desconecta
3. Estado inconsistente: El cliente y servidor pueden tener diferentes ideas sobre el estado de la conexión
4. Espectadores vs. Jugadores: Diferentes tipos de clientes requieren manejo diferente

Solución Encontrada

Se implementó un sistema de desconexión con callback que integra tres componentes:

1. Interfaz de Callback:

java

// GameClientHandler.java

```
public interface DisconnectCallback {  
    void onPlayerDisconnect(Integer gameNumber);  
}
```

```
private DisconnectCallback disconnectCallback;
```

```
public void setDisconnectCallback(DisconnectCallback callback) {  
    this.disconnectCallback = callback;  
}
```

2. Detección de Desconexión en el Handler:

java

// GameClientHandler.java

@Override

public void run() {

 // Thread para recibir comandos

 Thread inputThread = new Thread(() -> {

 while (this.connected) {

 try {

 String message = this.input.readUTF();

 processCommand(message);

 } catch (IOException e) {

 this.connected = false;

 SDL_Log("Client disconnected from Game " + gameNumber);

 }

 }

 });

 inputThread.start();

 // Loop principal del juego

 while (this.connected) {

 try {

 // Actualizar física y enviar estado...

 this.output.writeUTF(gameState);

 Thread.sleep(32);

 } catch (IOException e) {

 this.connected = false;

 }

```

// CRÍTICO: Cleanup al terminar

inputThread.join(1000); // Esperar max 1 segundo

if (!this.client.isClosed()) {
    this.client.close();
}

// Notificar desconexión

if (this.disconnectCallback != null) {
    this.disconnectCallback.onPlayerDisconnect(this.gameNumber);
}
}

```

3. Manejo de Desconexión en App:

```

java

// App.java

@Override

public void onPlayerDisconnect(Integer gameNumber) {
    System.out.println("Player disconnected from Game " + gameNumber);

    if (gameNumber == 1) {
        this.handler1 = null;

        this.game1.reset(); // Limpiar enemigos, coleccionables, resetear jugador

        System.out.println("Game 1 is now available for new players");
    } else if (gameNumber == 2) {
        this.handler2 = null;

        this.game2.reset();

        System.out.println("Game 2 is now available for new players");
    }
}

```

1.7. Conclusiones del Proyecto

1.7.1. Conclusiones Técnicas

Arquitectura Cliente-Servidor

La implementación de una arquitectura cliente-servidor autoritativa demostró ser la decisión correcta para este proyecto. Al mantener toda la lógica del juego en el servidor Java, se logró:

- **Consistencia garantizada:** Todos los clientes ven exactamente el mismo estado del juego
- **Prevención de cheating:** El cliente C solo envía inputs y recibe estados, sin poder manipular la lógica
- **Simplicidad en el cliente:** El cliente C se enfoca exclusivamente en renderizado e input
- **Escalabilidad:** El servidor puede manejar múltiples juegos simultáneos sin interferencia

Comunicación entre Lenguajes (Java ↔ C)

La interoperabilidad entre Java y C presentó desafíos significativos, especialmente en la serialización de datos:

- **Formato UTF de Java:** El uso de `writeUTF()` con su prefijo de longitud de 2 bytes requirió manejo especial en el cliente C
- **Endianness:** Aunque no fue un problema en localhost, es importante considerar que Java usa big-endian para los prefijos
- **Tipos de datos:** La conversión entre tipos de Java (Double, Integer) y C (float, int) requirió atención cuidadosa

Lección aprendida: En proyectos futuros, considerar formatos de serialización más estándares como JSON, Protocol Buffers o MessagePack, que tienen librerías bien probadas en múltiples lenguajes.

1.7.2. Logros del Proyecto

Funcionalidad completa:

- Juego multiplayer funcional con 2 salas simultáneas
- Sistema de espectadores completamente funcional

- Detección robusta de colisiones
- Manejo correcto de desconexiones

Calidad de código:

- Arquitectura limpia y mantenible
- Patrones de diseño aplicados correctamente
- Documentación exhaustiva
- Manejo robusto de errores

Performance:

- 30 FPS constantes en cliente y servidor
- Latencia < 50ms en red local
- Sin memory leaks detectados
- CPU usage razonable

1.8. Recomendaciones del Proyecto

1.8.1. Recomendaciones Técnicas

Para Proyectos de Comunicación de Red

1. Definir el protocolo claramente desde el inicio
2. Implementar logging desde el día 1
3. Usar herramientas de análisis de red
4. Implementar versioning del protocolo

Para Arquitecturas Cliente-Servidor

1. Servidor autoritativo siempre
2. Separación clara de responsabilidades
3. Rate limiting de comandos
4. Heartbeat/keepalive opcional pero recomendado

1.8.2. Recomendaciones de Gestión de Proyecto

1. Control de versiones disciplinado
2. Branches por feature
3. Code reviews (incluso en proyectos individuales)
4. Tracking de bugs sistematizado
5. Milestones claros

1.9. Bibliografía Consultada

Documentación Oficial

1. Oracle Java SE Documentation
2. SDL3 Documentation
3. SDL3_ttf Documentation

Libros

4. Glazer, Joshua & Madhav, Sanjay (2015) - Multiplayer Game Programming: Architecting Networked Games
5. Nystrom, Robert (2014) - Game Programming Patterns
6. Gamma, Erich et al. (1994) - Design Patterns: Elements of Reusable Object-Oriented Software
7. Stevens, W. Richard & Fenner, Bill & Rudoff, Andrew M. (2003) - UNIX Network Programming, Volume 1: The Sockets Networking API
8. Freeman, Eric & Robson, Elisabeth (2020) - Head First Design Patterns

